IMY 220

Practical 2

JavaScript & ES6

Due: Friday 31 July @ 08:20

The submission instructions are available on ClickUP. Any deviation from these instructions will result in a 10% deduction from your mark.

Instructions

- In this practical, you will use ES6 arrow functions and array methods to manipulate an array of PhotoShare post objects.
- You are not allowed to use loops. This includes:
    - `for`
    - `for...of`
    - `forEach`
    - `while`
    - `do...while`
- You have been provided with an array of data. You are not allowed to edit or remove any of the supplied data.
- All functions created for this practical must be arrow functions.
- You must use JavaScript array methods to process the supplied data.
- Download the `practical-2.zip` file from ClickUP. This ZIP file contains the base files for the practical.
- Do not modify the `index.html` file. This file will be used to display and check your output.
- You may add code to the `script.js` file, but you may not remove or edit any JavaScript code that has already been supplied.
- Include your student number as a comment at the top of the `script.js` file.
- Each task must produce the specified output using `console.log()`.

Tasks

1. Clean the post data

Create an arrow function called `cleanPosts`.

The function must:

a. Return a new array containing no `null` or `undefined` entries.
b. Not modify the original supplied array.
c. Be used as the starting point for all remaining tasks.
d. Use the filter() array method.
e. Call the function and log its returned array using console.log().

Example structure:

```
const cleanPosts = posts => {
    // Your code
};
const cleanedPosts = cleanPosts(posts);
console.log(cleanedPosts);
```

---

2. Create post summaries

Create an arrow function called `createPostSummaries`.

The function must:

a. Use the cleaned post data from Task 1.
b. Return a new array containing a summary object for every post.
c. Calculate the engagement of each post using:

```
engagement = likes + comments
```

Each summary object must have the following structure:

```
{
    id: 1,
    username: "leanne_dev",
    engagement: 46
}
```

d. Use destructuring when accessing the post properties.
e. Use the map() array method.
f. Call the function and log its returned array using `console.log()`.

Example structure:

```
const createPostSummaries = cleanedPosts => {
      // Your code
};

const postSummaries = createPostSummaries(cleanedPosts);
console.log(postSummaries);
```

---

3. Find the post with the highest engagement

Create an arrow function called `getHighestEngagementPost`.

The function must:

a. Accept the post summaries from Task 2 as a parameter.
b. Return the post summary with the highest engagement.
c. Use the reduce() array method.
d. Call the function and log its returned object using `console.log()`.

Example output:

```
{
    id: 3,
    username: "diffie",
    engagement: 57
}
```

Example structure:

```
const getHighestEngagementPost = postSummaries => {
    // Your code
};

console.log(getHighestEngagementPost(postSummaries));
```

---

4. Find posts at or below an engagement limit

Create an arrow function called `getLowEngagementPosts`.

The function must:

a. Accept an engagement limit as a parameter.
b. Use a default parameter value of `20`.
c. Return all post summaries with engagement less than or equal to the supplied limit.
d. Use the previously created postSummaries array from Task 2 within the function.
e. Use the filter() array method.
f. Call the function and log its returned array using `console.log()`.

Example structure:

```
const getLowEngagementPosts = (limit = 20) => {
      // Your code
};
console.log(getLowEngagementPosts());
console.log(getLowEngagementPosts(30));
```

---

5. Create post descriptions

Create an arrow function called `createPostDescriptions`.

The function must:

a. Accept the post summaries from Task 2 as a parameter.
b. Return a new array of strings in the following format:

```
leanne_dev received 46 total engagements.
```

c. Use template literals to create each string.
d. Use the `map()` array method.
e. Call the function and log its returned array using `console.log()`.

Example output:

```
[
    "leanne_dev received 46 total engagements.",
    "mark_codes received 11 total engagements.",
    "diffie received 57 total engagements.",
    "sarah received 21 total engagements.",
    "john received 37 total engagements."
]
```

Example structure:

```
const createPostDescriptions = postSummaries => {
```

```
    // Your code
};
console.log(createPostDescriptions(postSummaries));
```

---

6. Sort posts according to engagement

Create an arrow function called `sortPostsByEngagement`.

The function must:

a. Accept the post summaries from Task 2 as a parameter.
b. Return a new array containing the summaries sorted from highest engagement to lowest engagement.
c. Use the spread operator to prevent the original summary array from being modified.
d. Use the sort() array method.
e. Call the function and log its returned array using `console.log()`.

Example structure:

```
const sortPostsByEngagement = postSummaries => {
     // Your code
};
console.log(sortPostsByEngagement(postSummaries));
```

Submission Files

Submit only the following file:

- `script.js`

Submission Instructions

- ZIP the submission file using a tool such as 7-Zip or WinRAR.
- Name the ZIP file:

p2-{student_number}.zip

- Replace `{student_number}` with your student number.
- Submit the ZIP file before 08:20 to avoid a late submission.